



Draw It or Lose It
CS 230 Project Three Software Design
Hetal Ram
Version 1.2

Table of Contents

CS 230 Project Software Design Template	1
Table of Contents	2
Document Revision History	2
Executive Summary	3
Requirements	3
Design Constraints	3
System Architecture View	3
Domain Model	3
Evaluation	4
Recommendations	5

Document Revision History

Version	Date	Author	Comments
1.2	06/16/2026	Hetal Ram	Completed Project Three Software Design Document.

Executive Summary

The Gaming Room currently has Draw It or Lose It as an Android-only game and wants to expand it into a web-based application that can serve multiple platforms. The proposed solution is to develop a distributed web application that allows users to access the game from computers, tablets, and mobile devices through a browser. The application will support multiple games, teams, and players while keeping game and team names unique. The design uses object-oriented programming principles and design patterns, including Singleton and Iterator, to keep the system organized, reusable, and easier to maintain.

Requirements

The application must support one or more teams in a game, and each team must support multiple players. Game names and team names must be unique so users can check whether a name is already in use. The system must use unique identifiers for games, teams, and players. The application must also use a Singleton pattern for GameService so there is one shared service managing games. Iterator logic is used to search through games, teams, and players to prevent duplicate names.

Design Constraints

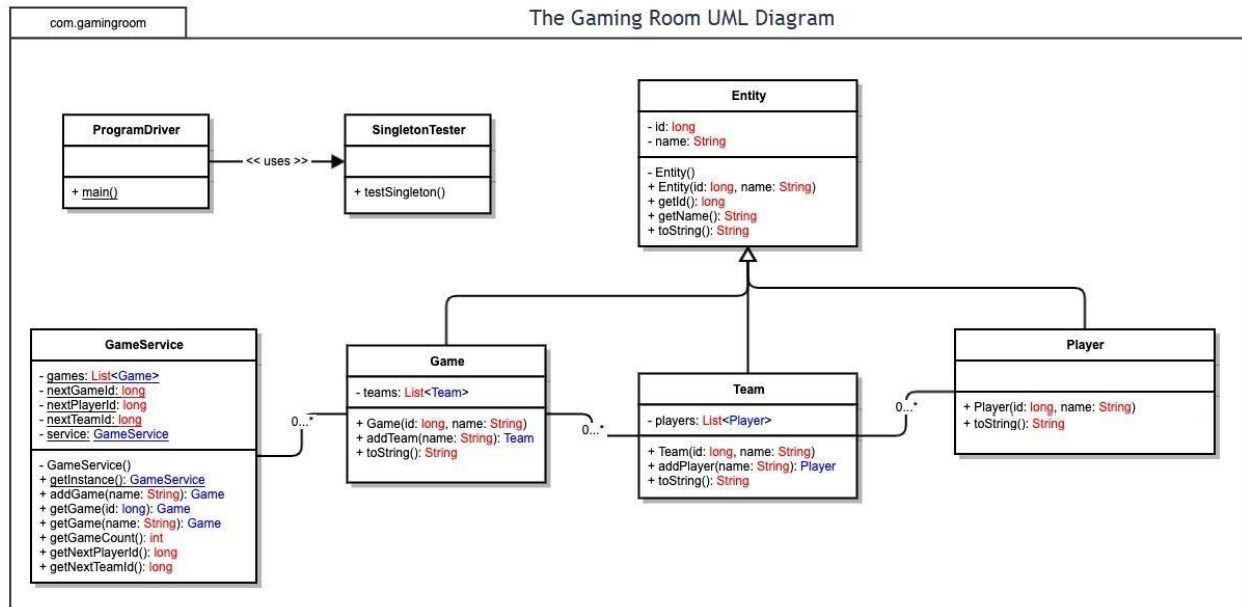
Because the game is being changed from an Android-only app to a web-based distributed application, the design must support multiple platforms, including Windows, Mac, Linux, and mobile devices. The application must work through a browser, so the client side should not depend on one operating system. The server side must handle multiple users, game sessions, teams, and players at the same time. The system must also protect user data, maintain unique names and IDs, and be scalable so more users can access the game in the future. These constraints affect development because the code must be modular, reusable, secure, and easy to test.

System Architecture View

The Draw It or Lose It application uses a web-based distributed architecture. Users access the application through client devices such as computers, tablets, and mobile phones using a web browser. The server side manages the game logic, player information, teams, and game sessions. The application stores game data and unique identifiers on the server side to ensure consistency across multiple platforms and users.

Domain Model

The domain model shows the main classes used in the Draw It or Lose It application. The Entity class is the base class that stores common fields, including id and name. Game, Team, and Player inherit from Entity, which demonstrates inheritance and helps reduce repeated code. The GameService class manages the list of games and uses the Singleton pattern, so only one service instance controls the game registry and ID creation. A Game can have multiple Team objects, and each Team can have multiple Player objects. The Iterator pattern is used to search through lists of games, teams, and players to check for duplicate names. The model also demonstrates encapsulation because fields are private and accessed through public methods.



Evaluation

Using your experience to evaluate the characteristics, advantages, and weaknesses of each operating platform (Linux, Mac, and Windows) as well as mobile devices, consider the requirements outlined below and articulate your findings for each. As you complete the table, keep in mind your client's requirements and look at the situation holistically, as it all has to work together.

Development Requirements	Mac	Linux	Windows	Mobile Devices
Server Side	Mac can host a web-based application, but it is not the best choice for serverside deployment because Apple hardware and support costs are higher. It is stable and secure, but scaling a production web server on Mac may be more expensive than Linux.	Linux is a strong choice for serverside hosting because it is stable, secure, cost-effective, and widely used for web servers. It supports Java applications well and can scale as more users access Draw It or Lose It.	Windows can host a web-based application and is familiar to many developers. However, licensing costs may be higher, and it may require more system resources than Linux for server hosting.	Mobile devices are not appropriate for hosting the server side of this application. They are best used as client devices because they have limited processing power, storage, and reliability compared to a server.

Client Side	Mac users can access the game through a modern web browser, so little platform-specific development is needed. Testing should still be completed on Safari and other browsers to ensure compatibility.	Linux users can access the game through browsers such as Chrome or Firefox. Development costs are low because a web-based design avoids creating a separate Linux application.	Windows users can access the game through common browsers such as Chrome, Edge, or Firefox. Because Windows is widely used, browser testing on Windows is important for client-side compatibility.	Mobile users can access the game through a browser on phones and tablets. The interface must be responsive, easy to touch, and tested on different screen sizes.
Development Tools	Developers on Mac can use Java, Eclipse, Git, and browser testing tools. Mac also supports web development tools for testing the application in Safari and other browsers.	Developers on Linux can use Java, Eclipse or IntelliJ IDEA, Git, Apache or Nginx, and database tools such as MySQL or PostgreSQL. Most of these tools are open source, which reduces development costs.	Developers on Windows can use Java, Eclipse, Git, and browser testing tools. Windows is a good development environment because it is familiar and supports the tools needed for this Java application.	Mobile testing requires browser developer tools, responsive design testing, and real device testing on Android and iOS. The application should be tested for touch controls, screen size, and performance.

Recommendations

Analyze the characteristics of and techniques specific to various systems architectures and make a recommendation to The Gaming Room. Specifically, address the following:

1. **Operating Platform:** I recommend Linux as the server operating platform for Draw It or Lose It. Linux is stable, secure, cost-effective, and commonly used for hosting web-based applications. Linux also supports popular web servers such as Apache and Nginx and has low licensing costs because many Linux distributions are open source. It also supports Java applications and can serve users on Windows, Mac, Linux, and mobile devices through a browser.
2. **Operating Systems Architectures:** The recommended architecture is a Linux-based server environment using a client-server model. The server manages the game logic, game sessions, teams, players, and stored data. Client devices connect through a web browser, which allows the application to support multiple operating systems without building a separate application for each one.

3. **Storage Management:** The application should use server-side storage with a relational database to store users, games, teams, players, and unique identifiers. This helps keep data organized, consistent, and easy to retrieve when multiple users are playing.
4. **Memory Management:** Linux uses memory management techniques such as virtual memory and process isolation to keep applications stable. For Draw It or Lose It, the server should manage active game sessions carefully and release memory when games end so the application continues to perform well.
5. **Distributed Systems and Networks:** Draw It or Lose It should use a distributed client-server system. Users connect from different platforms through the internet, and the server handles shared game data and communication. The design should account for network delays, outages, and connection problems by saving important data on the server and handling reconnections when possible.
6. **Security:** The application should protect user information with secure login, strong password handling, encrypted communication using HTTPS, and role-based access where needed. Server-side validation should be used to protect the system from invalid input, and regular updates should be applied to keep the Linux server secure.